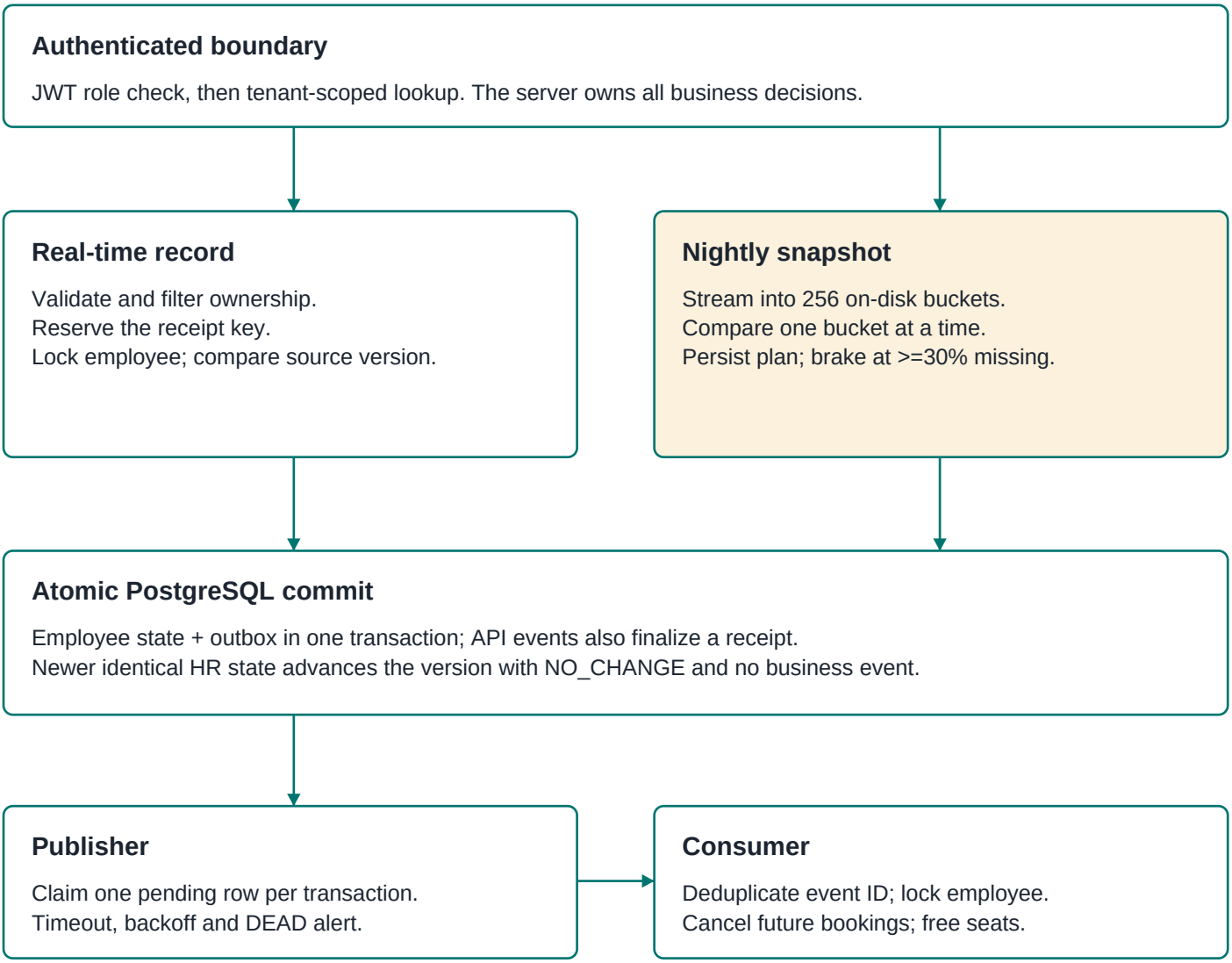
Roster Sync: state before delivery

Idempotent HR events, out-of-order versions and safe nightly reconciliation. One application and one PostgreSQL primary; the React console shares the API origin.



Identity and ownership

Employee key: tenant + external ID. Receipt key: tenant + employee + source version. CREATE/UPDATE carry full HR state. HR input never overwrites pickup point or special assistance; invalid rows do not consume a key.

Bounded work

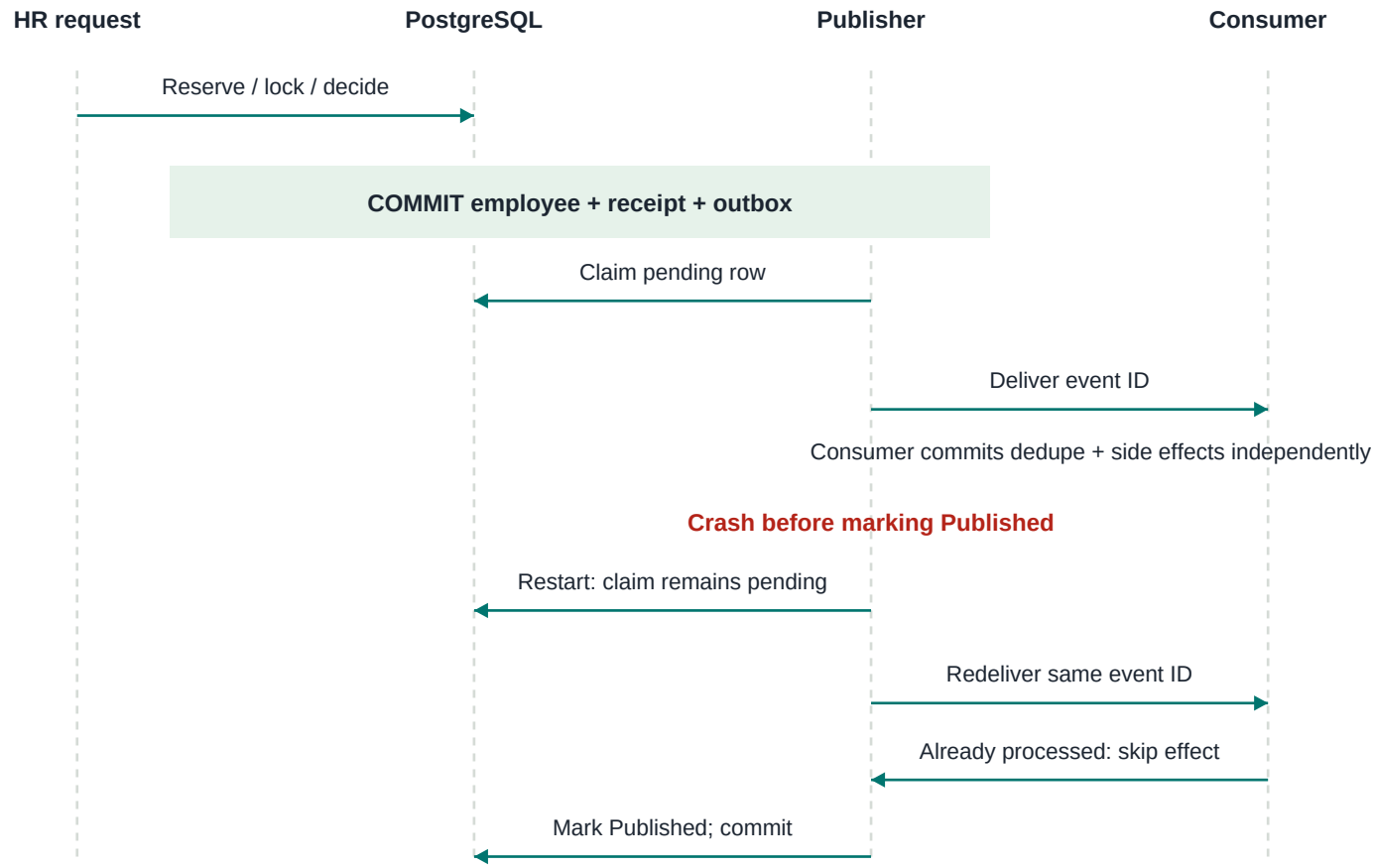
Comparison: O(N + M) time, expected O((N + M)/256) heap, O(N) spill disk. Skew sets the largest bucket. A brake stops all apply until an operator decides. Apply locks only the relevant employee rows in 40-action chunks and rechecks expected versions.

Explainable core

api -> application -> pure Java domain. Infrastructure supplies JPA/JDBC, security, cache and metrics. Domain rules have no Spring, Jakarta or SQL dependencies. Node exists only in the static build stage.

Hardest flow: crash after delivery

The consumer can commit before publication is acknowledged. Retrying execution is necessary; idempotent, state-aware effects make that retry safe.



Failure-mode table

Failure	Visible behavior	Recovery
Invalid / replay / stale	Per-record rejection or safe no-op	Correct and resend; receipts/version prevent regression.
Truncated file	Missing count, alert, no apply	Operator approves all, skips exits or rejects.
Crash before commit	No partial employee/event/receipt	Rollback; replay the same input.
Crash after commit / delivery	Pending event; possible redelivery	Restart publisher; consumer dedupe/current-state check.
Sink timeout / failure	Backoff, then DEAD + one alert	Manual retry; later pending rows continue.
Interrupted recon	Failed comparison or pending actions	Re-upload comparison; restart resumes apply.
Database unavailable	Safe dependency error; fail closed	No speculative writes; resume after store recovery.

Trade-offs: single instance/primary and local upload storage simplify consistency. Multi-instance operation needs leases and shared storage. Caffeine staleness is bounded (office 10 min; employee 30 s), with after-commit eviction. JWTs are stateless and remain valid until expiry after account disablement.

Full schema, API, algorithm, evidence and decision records: docs/DESIGN.md and its linked documents.